6. Interfícies d'usuari

2025-2026

Continguts

- ✓ Llibreries gràfiques.
- ✓ Contenidors Swing.
- ✓ Layouts.
- ✓ Apariència de les finestres.
- ✓ Events.

1. Llibreries gràfiques en Java

Per poder aplicar interfícies gràfiques amb Java, és necessari fer ús d'eines o **llibreries externes** a Java.

Les dues més conegudes son:

- **AWT (Abstract Window Toolkit)** - Kit d'eines abstractes per finestres
- **SWING**

AWT (Abstract Window Toolkit)

És part de la **JFC** (Java Foundation Classes) - Classes base de Java.

Es pot definir com una **biblioteca de classes per desenvolupar IGU** (Interfícies Gràfiques d'Usuari, GUI en anglès).

Els **Components** permeten a l'usuari **interactuar amb la aplicació**. Tots els components amb AWT seran **instàncies de la classe Component** o de classes filla seves.

SWING

Apareix com a **una millora front AWT**.

Els **components** emprats amb la llibreria de SWING estan programats amb codi no natiu. Això permet que siguin **més portables**.
A més, aquests components són **més potents** que els anteriors de AWT i **s'identifiquen amb una "J"** abans del seu nom. Per exemple "JButton".

Es recomana emprar SWING en comptes de AWT.

Exemple Java SWING

```java
import javax.swing.*;

public class ProvaSwing {

    public static void main(String[] args) {

        JFrame frame = new JFrame("Ventana");
        frame.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
        JLabel label = new JLabel("Hola Mundo");
        frame.getContentPane().add(label);
        frame.pack();
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

2. Contenidors SWING d'alt nivell

JFrame - Implementa una finestra d'aplicació.

JDialog - Implementa una finestra de tipus diàleg. Són com a finestres secundàries.

JApplet - Són com una aplicació Java que s'executa dins d'un navegador web a la màquina client.

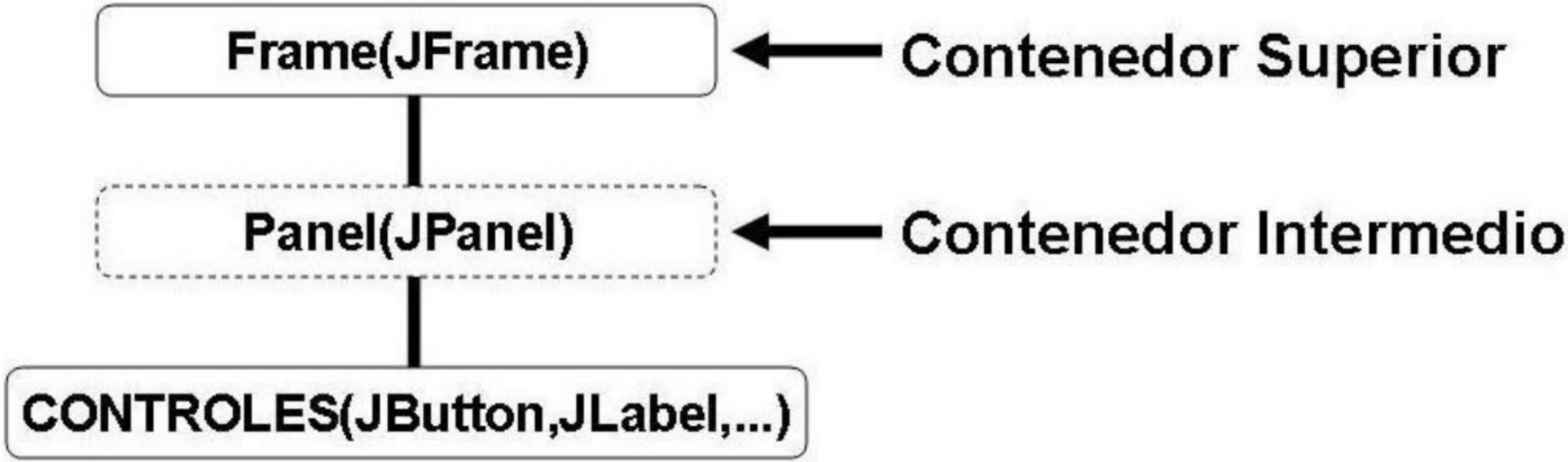
Exemple amb JDialog

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class ExempleJDialog {
    public static void main(String[] args) {
        // Creem el JFrame principal
        JFrame finestra = new JFrame("Exemple de JDialog");
        finestra.setSize(400, 200);
        finestra.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        finestra.setLayout(new FlowLayout());

        // Botó per obrir el JDialog
        JButton botoObrir = new JButton("Obrir diàleg");
        finestra.add(botoObrir);

        // Afegim un listener per obrir el JDialog quan es premi
        botoObrir.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                mostrarDialeg(finestra);
            }
        });

        // Fem visible la finestra principal
        finestra.setVisible(true);
    }

    // Mètode per crear i mostrar un JDialog
    private static void mostrarDialeg(JFrame parent) {
        // Creem el diàleg modal (bloqueja la finestra principal fins que es tanqui)
        JDialog dialeg = new JDialog(parent, "Missatge Important", true);
        dialeg.setSize(300, 150);
        dialeg.setLayout(new BorderLayout());

        // Afegim un missatge
        JLabel etiqueta = new JLabel("Això és un diàleg!", JLabel.CENTER);
        dialeg.add(etiqueta, BorderLayout.CENTER);

        // Botó per tancar el diàleg
        JButton botoTancar = new JButton("D'acord");
        botoTancar.addActionListener(e -> dialeg.dispose()); // Tanca el diàleg
        dialeg.add(botoTancar, BorderLayout.SOUTH);

        // Fem visible el diàleg
        dialeg.setLocationRelativeTo(parent); // Centra el diàleg respecte a la finestra principal
        dialeg.setVisible(true);
    }
}
```

Contenidors Swing

JPanel - Són contenidors de components lleugers.

Els JPanel, poden contenir altres panells.



Contenidors Swing

1. `JFrame` (Ventana Principal)

- Es una ventana independiente que puede contener otros componentes dentro de ella.
- Hereda de `java.awt.Frame` y proporciona una estructura completa de ventana con una barra de título, bordes y botones como minimizar, maximizar y cerrar.
- Se usa comúnmente como la ventana principal de una aplicación gráfica.
- Debe configurarse con `setSize()` y `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)` para definir su tamaño y su comportamiento al cerrarse.

```java
import javax.swing.*;

public class VentanaEjemplo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Ejemplo de JFrame");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Contenidors Swing

2. `JPanel` (Panel de Contenido)

- Es un contenedor **flexible** que se usa dentro de un `JFrame` o dentro de otros `JPanel`.
- Sirve para organizar y agrupar componentes como botones, etiquetas y cuadros de texto.
- **No es una ventana independiente;** necesita estar dentro de un `JFrame` o de otro contenedor.
- Puede personalizarse con colores, bordes y diferentes disposiciones (`LayoutManagers`).

```java
import javax.swing.*;
import java.awt.*;

public class PanelEjemplo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Ejemplo de JPanel");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Crear un JPanel
        JPanel panel = new JPanel();
        panel.setBackground(Color.LIGHT_GRAY);

        // Agregar un botón al panel
        JButton boton = new JButton("Haz clic");
        panel.add(boton);

        // Agregar el panel al frame
        frame.add(panel);

        frame.setVisible(true);
    }
}
```

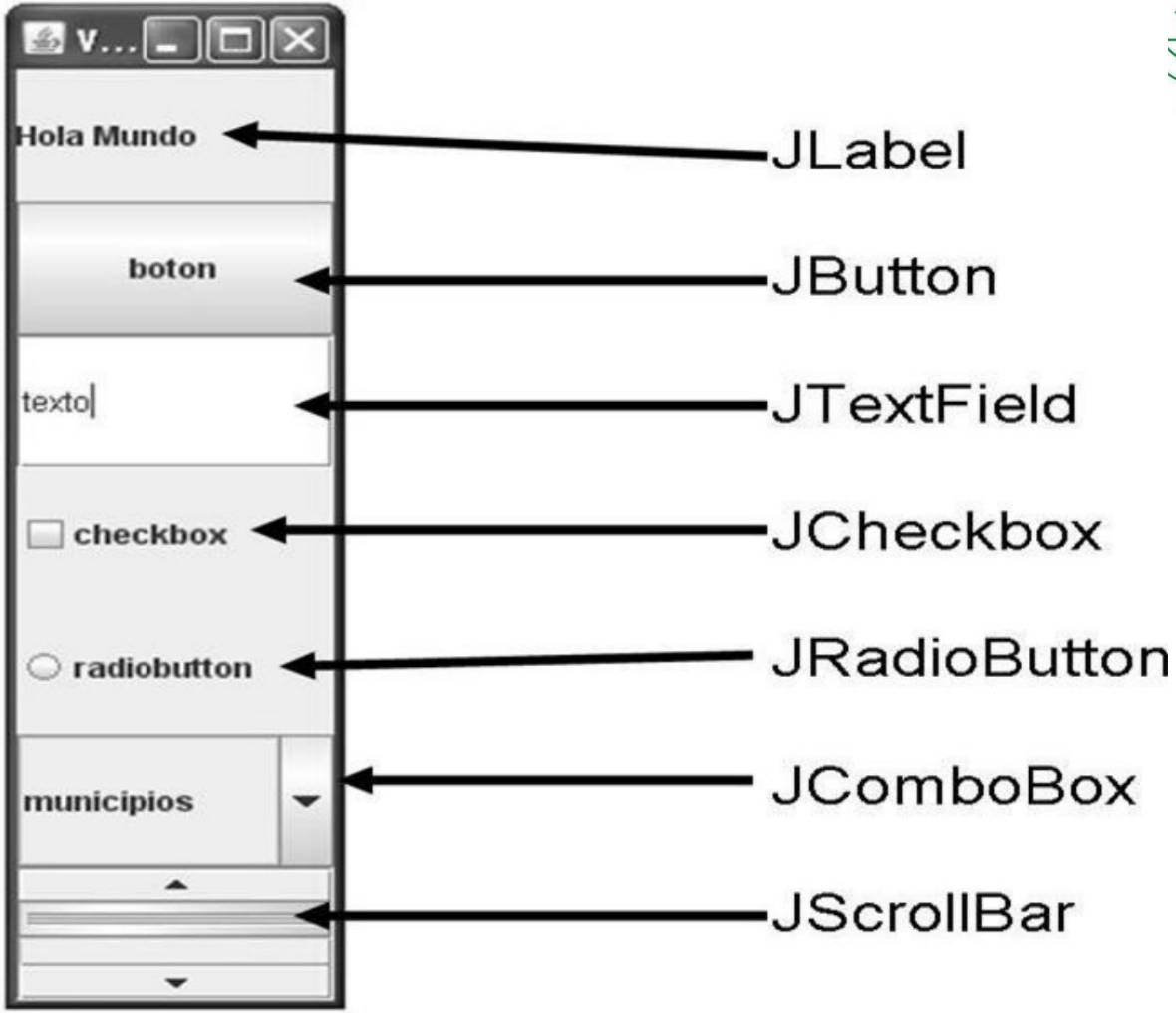
Contenidors Swing

Diferencias Clave

Característica	`JFrame`	`JPanel`
Es un contenedor independiente	✅ Sí	❌ No
Puede tener barra de título, minimizar y cerrar	✅ Sí	❌ No
Se usa como ventana principal	✅ Sí	❌ No
Puede contener otros componentes	✅ Sí	✅ Sí
Se usa para organizar componentes dentro de una ventana	❌ No	✅ Sí

Components Swing

Objecte	Descripció
JButton	Botó estàndard
JLabel	Etiqueta de text estàndard
JTextField	Quadre de text
JTextArea	Quadre de text multilinea
JCheckBox	Checkbox o casilla de verificació
JRadioButton	RadioButton o botons d'opció
JComboBox	Llista desplegable
JScrollBar	Barra de desplaçament


V...
Hola Mundo
boton
texto
checkbox
radiobutton
municipios
JLabel
JButton
JTextField
JCheckbox
JRadioButton
JComboBox
JScrollBar

3. Organització dels controladors a un contenidor

Els **Layout Manager** ens permeten **administrar el disseny**.

- Per tant, per organitzar els objectes gràfics necessitarem un objecte que implementi la interfície LayoutManager. (Per exemple un JPanel)
- Els Layout Manager es poden establir al crear un objecte (Constructor)

Exemple amb LayoutManager

```java
import javax.swing.*;
import java.awt.*;

public class ExempleLayout {
    public static void main(String[] args) {
        // Creem la finestra principal (JFrame)
        JFrame finestra = new JFrame("Exemple de LayoutManager");
        finestra.setSize(500, 300);
        finestra.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        finestra.setLayout(new BorderLayout()); // Establim BorderLayout per al JFrame

        // Creem un panell superior amb FlowLayout
        JPanel panellSuperior = new JPanel();
        panellSuperior.setLayout(new FlowLayout()); // Organitza els elements en línia
        panellSuperior.add(new JButton("Botó 1"));
        panellSuperior.add(new JButton("Botó 2"));
        panellSuperior.add(new JButton("Botó 3"));

        // Creem un panell central amb GridLayout
        JPanel panellCentral = new JPanel();
        panellCentral.setLayout(new GridLayout(2, 2)); // 2 files, 2 columnes
        panellCentral.add(new JLabel("Etiqueta 1"));
        panellCentral.add(new JTextField("Text 1"));
        panellCentral.add(new JLabel("Etiqueta 2"));
        panellCentral.add(new JTextField("Text 2"));

        // Afegim els panells a la finestra
        finestra.add(panellSuperior, BorderLayout.NORTH);
        finestra.add(panellCentral, BorderLayout.CENTER);

        // Fem visible la finestra
        finestra.setVisible(true);
    }
}
```

Tipus de LayoutManager

- **FlowLayout** - Posa els components al contenidor de l'esquerra a dreta. És el layout Manager per defecte als panells.
- **BorderLayout** - Divideix el contenidor en 5 parts (nord, sud, oest, centre, est)
- **CardLayout** - Permet posar grups de components diferents en moments diferents de l'execució del programa.
- **GridLayout** - Posa els components en fileres i columnes.
- **GridBagLayout** - Posa els components en fileres i columnes, però un component pot ocupar més d'un columna.
- **BoxLayout** - Posa els components en una filera o columna ajustant-se.

4. Apariència de les finestres

Els estils de les finestres **poden ser modificats** mitjançant una sèrie de **Look and Feel** genèrics.

Java té un **gestor** de la IGU anomenat **UIManager** que controla la visualització genèrica de les finestres i dels components que les formen.

Aquest UIManager es troba a la classe javax.swing.UIManager.

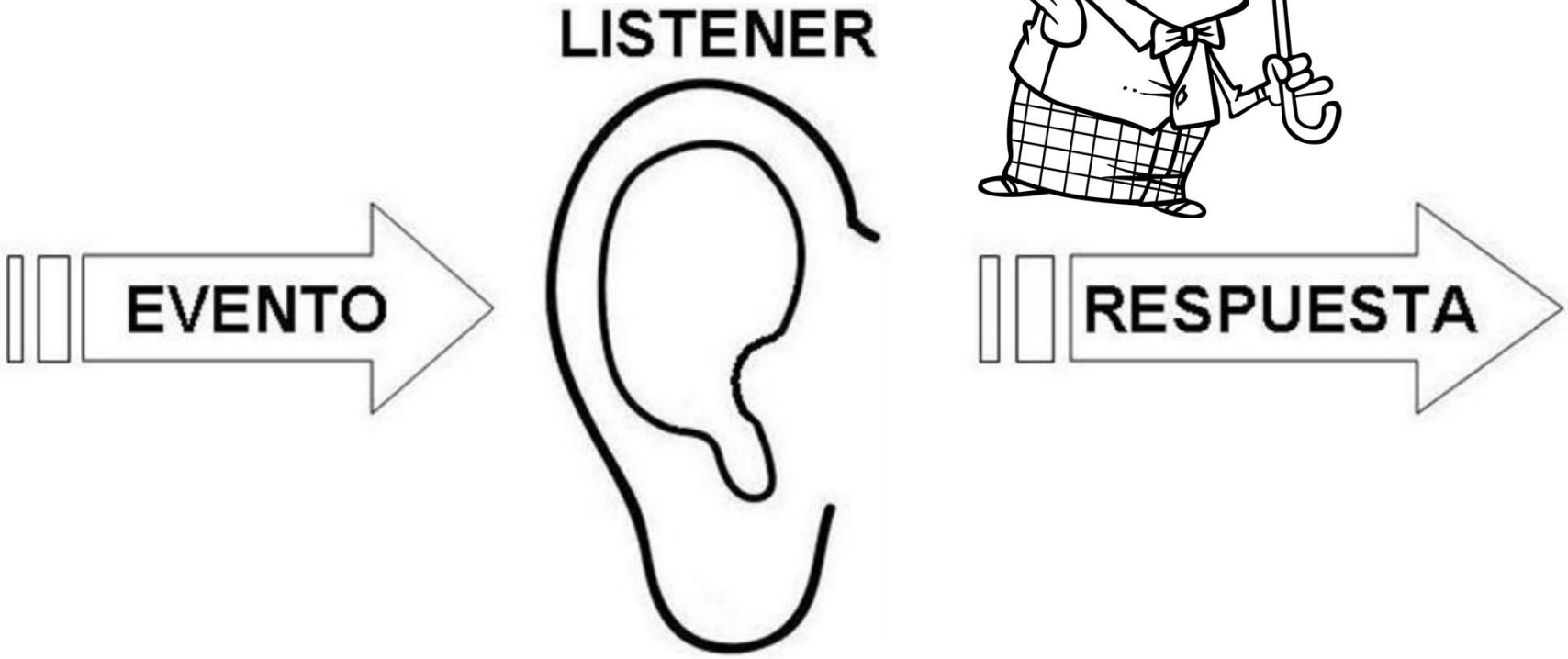
Establint un Look & Feel

```
//Establecer el look and feel metal.
//Un L&F muy típico de Java.
try {
UIManager.setLookAndFeel("javax.swing.plaf.metal.MetalFileChooserUI");
}catch (Exception e) {
    e.printStackTrace();
}
```

Establint un Look & Feel

Cerca looks & feels i prova-ho

5. Concepte d'Event



Events i controladors d'Events

- Existeixen una sèrie de controladors d'events (listeners) els quals han d'estar associats al component per que s'executi la resposta necessària.

- Aquests listeners són diferents depenent dels events als que han de donar resposta.

- Els listeners estan especialitzats depenent de l'event actual.

Listeners associats a components

Listener	Components	Acció que responen
ActionListener	JButton JTextField JComboBox…	Pressionar el botó Fer intro Triar una opció….
AdjustmentListener	JScrollBar…	Moure la barra de desplaçament
FocusListener	JButton JTextField JComboBox…	Les accions d'aquest listener son obtenir i perdre el focus (ens posa en el component i ens marxem quan estava actiu)
ItemListener	JCheckBox…	Seleccionar i deseleccionar l'opció

Listeners associats a components (II)

Listener	Components	Acció que responen
KeyListener	JTextArea JTextField…	Pressionar una tecla quan el component té el focus
MouseListener	Múltiples components	Accions com pressionar el botó del ratolí
MouseMotionListener	Múltiples components	Accions com arrossegar (drag) o pasar per damunt un objecte
WindowListener	JFrame	Accions relatives a la finestra com per exemple tancar-la

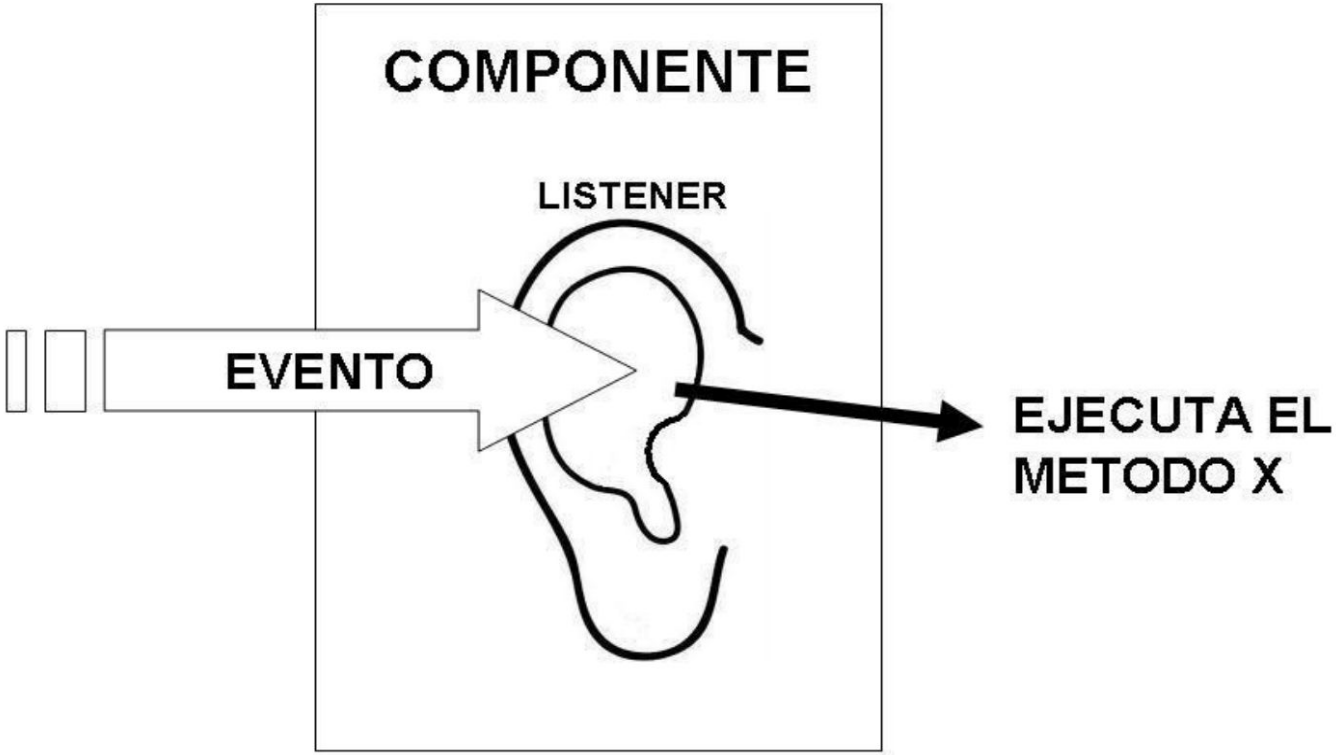
Per emprar Events...

1. Es crea el component

1. S’afegeix el listener adequat al component i el listener escoltarà l’acció sobre el component

1. Dependent del component o l’acció s’executarà l’acció necessària.

Per emprar Events...



Exemple de JButton amb Listener

```
// Configuracion del JButton
JButton button = new JButton("Generar nuevo numero");
button.setAlignmentX(Component.CENTER_ALIGNMENT);
button.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent ev) {
        Random random = new Random();
        Integer numero = random.nextInt(100);
        label.setText(numero.toString());
    }
});
```

```java
package es.cide.programacio;

import javax.swing.*; // Importem la llibreria Swing per crear la interfície gràfica
import java.awt.*; // Importem la llibreria per gestionar el disseny
import java.awt.event.*; // Importem la llibreria per gestionar els esdeveniments

public class AplicacioSwing {
    Run | Debug
    public static void main(String[] args) {
        JFrame finestra = new JFrame(title:"Exemple Swing"); // Creem la finestra principal
        finestra.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Configurem la finestra per tancar l'aplicació
        finestra.setSize(width:300, height:150); // Establim la mida de la finestra

        JPanel panel = new JPanel(); // Creem un JPanel per organitzar els components
        panel.setLayout(new GridLayout(rows:3, cols:1)); // Establim un disseny de graella amb 3 files i 1 columna

        JTextField campText = new JTextField(columns:15); // Creem un camp de text
        panel.add(campText); // Afegim el camp de text al panell

        JLabel etiqueta = new JLabel(text:"Aquí irá el texto..."); // Creem una etiqueta per mostrar el text
        panel.add(etiqueta); // Afegim l'etiqueta al panell

        JButton boto = new JButton(text:"Mostrar text"); // Creem un botó
        panel.add(boto); // Afegim el botó al panell

        boto.addActionListener(new ActionListener() { // Afegim un gestor d'esdeveniments al botó
            @Override
            public void actionPerformed(ActionEvent e) {
                etiqueta.setText(campText.getText()); // Copiem el text a l'etiqueta
            }
        });

        finestra.add(panel); // Afegim el panell a la finestra
        finestra.setVisible(b:true); // Fem visible la finestra principal
    }
}
```

Funcions Lambda

Una funció *lambda* en Java (introduïda a Java 8) és una funció anònima —sense nom— que permet escriure codi més compacte i funcional, ja que es tracta com un objecte que es pot passar com a argument o emmagatzemar en una variable. Implementen interfícies funcionals (interfícies amb un sol mètode abstracte).

Sintaxi i característiques clau:

Format: (paràmetres) -> { cos }.

```
() -> System.out.println("Hola Mundo");
```

Components:

Paràmetres: Accepta arguments, o parèntesis buits () si no en té.
Fletxa: Operador ->.
Cos: Expressió o bloc de codi.
Concisions: Sovint s'ometen els tipus de dades i les claus si l'expressió és simple.

Exemples:

`s -> s.length()` (Rep un String, retorna la seva longitud).
`(a, b) -> a + b` (Suma dos nombres).
`() -> System.out.println("Hola")` (No rep arguments).

```
(a, b) -> {
    int resultado = a + b;
    return resultado;
};
```

Exemple amb Lambda

```java
package es.cide.programacio;

import javax.swing.*;

public class LambdaActionListenerExample {
    Run | Debug
    public static void main(String[] args) {
        JFrame frame = new JFrame(title:"Ejemplo Lambda");
        JButton button = new JButton(text:"Haz clic");

        // Usando una expresión lambda para el ActionListener
        button.addActionListener(e -> JOptionPane.showMessageDialog(frame, message:"¡Botón presionado!"));

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(width:300, height:200);
        frame.setLayout(manager:null);

        button.setBounds(x:100, y:70, width:100, height:30); // Posición y tamaño del botón
        frame.add(button);

        frame.setVisible(b:true);
    }
}
```

Java Swing: Elements gràfics simples

En Java Swing, pots dibuixar elements gràfics utilitzant la classe `Graphics` i `Graphics2D` dins del mètode `paintComponent()` d'un `JPanel`. Els elements més rellevants per dibuixar en una pantalla són:

1. **Línies** → `g.drawLine(x1, y1, x2, y2);`
2. **Rectangles i quadrats** → `g.drawRect(x, y, amplada, altura);` (contorn) o `g.fillRect(x, y, amplada, altura);` (ple)
3. **Cercles i el·lipses** → `g.drawOval(x, y, amplada, altura);` o `g.fillOval(x, y, amplada, altura);`
4. **Arcs** → `g.drawArc(x, y, amplada, altura, angleInicial, angleFinal);`
5. **Polígons** → `g.drawPolygon(int[] xPoints, int[] yPoints, int numPoints);`
6. **Text** → `g.drawString("Hola, món!", x, y);`
7. **Imatges PNG o JPEG** → `g.drawImage(imatge, x, y, this);` amb `ImageIcon` i `Image`

Java Swing: Elements gràfics simples

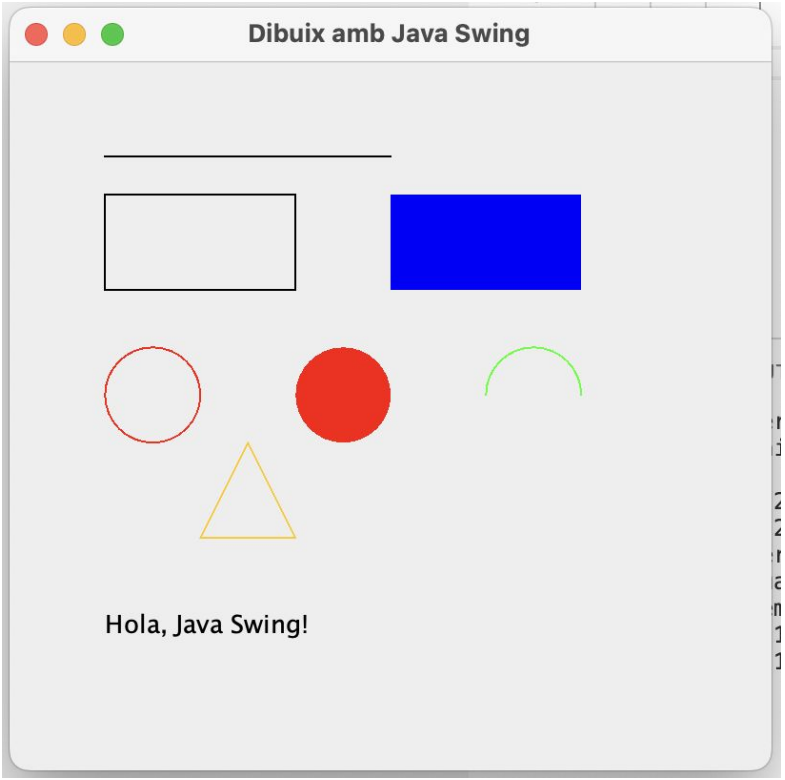
La diferència principal entre `Graphics` i `Graphics2D` és que `Graphics2D` és una extensió més avançada de `Graphics`, que ofereix més funcionalitats per a dibuixar i transformar gràfics amb més control i qualitat.

Diferències clau:

1. **Més funcionalitats**:
   `Graphics2D` permet transformacions afegides, com ara rotació, escalat i translació de formes.
2. **Control de qualitat**:
   Permet ajustar l'antialiasing per obtenir dibuixos més suaus (`g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);`).
3. **Transformacions avançades**:
   Pots aplicar matrius de transformació (`AffineTransform`) per canviar la forma d'un objecte.
4. **Dibuix de traços personalitzats**:
   Amb `Graphics2D`, pots definir gruixos i estils de línies (`BasicStroke`).

Nosaltres usarem directament Graphics2D

Java Swing: Elements gràfics simples



```java
package com.example;

import javax.swing.*;
import java.awt.*;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        JFrame finestra = new JFrame(title:"Dibuix amb Java Swing");
        JPanel panel = new JPanel() {
            private Image imatge = new ImageIcon(filename:"imatge.png").getImage();

            @Override
            protected void paintComponent(Graphics g) {
                super.paintComponent(g);
                Graphics2D g2d = (Graphics2D) g;

                // Dibuixar una línia
                g2d.drawLine(x1:50, y1:50, x2:200, y2:50);

                // Dibuixar un rectangle
                g2d.drawRect(x:50, y:70, width:100, height:50);

                // Dibuixar un rectangle ple
                g2d.setColor(Color.BLUE);
                g2d.fillRect(x:200, y:70, width:100, height:50);

                // Dibuixar un cercle (oval)
                g2d.setColor(Color.RED);
                g2d.drawOval(x:50, y:150, width:50, height:50);

                // Dibuixar un cercle ple
                g2d.fillOval(x:150, y:150, width:50, height:50);

                // Dibuixar un arc
                g2d.setColor(Color.GREEN);
                g2d.drawArc(x:250, y:150, width:50, height:50, startAngle:0, arcAngle:180);

                // Dibuixar un polígon (triangle)
                int[] xPoints = {100, 125, 150};
                int[] yPoints = {250, 200, 250};
                g2d.setColor(Color.ORANGE);
                g2d.drawPolygon(xPoints, yPoints, nPoints:3);

                // Dibuixar text
                g2d.setColor(Color.BLACK);
                g2d.drawString(str:"Hola, Java Swing!", x:50, y:300);

                // Dibuixar una imatge (ha d'existir imatge.png)
                g2d.drawImage(imatge, x:200, y:250, this);
            }
        };

        finestra.add(panel);
        finestra.setSize(width:400, height:400);
        finestra.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        finestra.setVisible(b:true);
    }
}
```

Java Swing + un objecte Timer

El `Timer` de Swing (`javax.swing.Timer`) és un objecte que permet executar accions de manera repetitiva amb un cert retard entre cada execució. S'utilitza habitualment per animacions o tasques programades en aplicacions gràfiques de Java.

Com funciona el `Timer`?

Quan es crea un objecte de tipus `Timer`, aquest rep dos paràmetres principals:

1. **Retràs en mil·lisegons** – Defineix cada quant temps s'ha d'executar l'acció.
2. **ActionListener** – Indica l'objecte que rebrà l'esdeveniment i executarà una acció.

Un cop creat, el `Timer` es pot iniciar, aturar o reiniciar segons es necessiti.

Exemple amb Timer

```java
package com.example;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class Comptador extends JFrame {

    private JLabel etiquetaComptador; // Etiqueta per mostrar el comptador
    private int comptador = 0; // Variable per mantenir el valor del comptador
    private Timer timer; // Objecte Timer per actualitzar el comptador

    public Comptador() {
        // Configuració de la finestra
        setTitle(title:"Comptador");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(width:200, height:100);
        setLayout(new FlowLayout());

        // Creació de l'etiqueta per mostrar el comptador
        etiquetaComptador = new JLabel(text:"0");
        etiquetaComptador.setFont(new Font(name:"Arial", Font.BOLD, size:24));
        add(etiquetaComptador);

        // Creació del Timer per actualitzar el comptador cada segon
        timer = new Timer(delay:1000, new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                // Incrementa el comptador i actualitza l'etiqueta
                comptador++;
                etiquetaComptador.setText(String.valueOf(comptador));
            }
        });

        // Inicialitza el Timer
        timer.start();

        // Mostra la finestra
        setVisible(b:true);
    }

    Run | Debug
    public static void main(String[] args) {
        new Comptador();
    }
}
```

Graphics2D + Timer

```java
package com.example;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

// Classe que representa un panell on es dibuixa un cercle que rebota
public class CercleRebotant extends JPanel implements ActionListener {
    private int x = 50, y = 50; // Coordenades inicials del cercle
    private int dx = 2, dy = 2; // Velocitat del moviment en X i Y
    private final int RADI = 20; // Radi del cercle
    private final int DELAY = 10; // Retard del temporitzador en mil·lisegons
    private Timer timer; // Temporitzador per controlar l'animació

    // Constructor que inicialitza el panell i inicia el temporitzador
    public CercleRebotant() {
        setBackground(Color.WHITE); // Defineix el color de fons del panell
        timer = new Timer(DELAY, this); // Crea el temporitzador amb retard especificat
        timer.start(); // Inicia el temporitzador
    }

    // Mètode per dibuixar el cercle dins del panell
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2d = (Graphics2D) g; // Conversió a Graphics2D per millorar el dibuix
        g2d.setColor(Color.RED); // Defineix el color del cercle
        g2d.fillOval(x, y, RADI * 2, RADI * 2); // Dibuixa el cercle amb les coordenades i el radi
    }

    // Mètode que s'executa a cada tic del temporitzador per moure el cercle
    @Override
    public void actionPerformed(ActionEvent e) {
        // Comprova si el cercle toca les vores horitzontals
        if (x + 2 * RADI >= getWidth() || x <= 0) {
            dx = -dx; // Inverteix la direcció horitzontal
        }
        // Comprova si el cercle toca les vores verticals
        if (y + 2 * RADI >= getHeight() || y <= 0) {
            dy = -dy; // Inverteix la direcció vertical
        }
        // Actualitza la posició del cercle
        x += dx;
        y += dy;
        repaint(); // Redibuixa el panell per actualitzar la posició del cercle
    }

    // Mètode principal per iniciar l'aplicació
    Run | Debug
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame(title:"Cercle Rebotant"); // Crea la finestra
            CercleRebotant panel = new CercleRebotant(); // Crea una instància del panell
            frame.add(panel); // Afegeix el panell a la finestra
            frame.setSize(width:400, height:300); // Defineix la mida de la finestra
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Configura el tancament de la finestra
            frame.setLocationRelativeTo(c:null); // Centra la finestra a la pantalla
            frame.setVisible(b:true); // Mostra la finestra
        });
    }
}
```